

HOSTED BY



ELSEVIER

Contents lists available at ScienceDirect

Journal of King Saud University - Computer and Information Sciences

journal homepage: www.sciencedirect.com

Full Length Article

High-throughput systolic array-based accelerator for hybrid transformer-CNN networks

Qingzeng Song^a, Yao Dai^{a,*}, Hao Lu^a, Guanghao Jin^b^a School of Computer Science and Technology, Tiangong University, Tianjin, China^b Beijing Polytechnic, Beijing, China

ARTICLE INFO

Keywords:

Hardware accelerator
Hybrid transformer-CNN
Systolic array
FPGA

ABSTRACT

In this era of Transformers enjoying remarkable success, Convolutional Neural Networks (CNNs) remain highly relevant and useful. Indeed, hybrid Transformer-CNN network architectures, which combine the benefits of both approaches, have achieved impressive results. Vision Transformer (ViT) is a significant neural network architecture that features a convolutional layer as its first layer, primarily built on the transformer framework. However, owing to the distinct computation patterns inherent in attention and convolution, existing hardware accelerators for these two models are typically designed separately and lack a unified approach toward accelerating both models efficiently. In this paper, we present a dedicated accelerator on a field-programmable gate array (FPGA) platform. The accelerator, which integrates a configurable three-dimensional systolic array, is specifically designed to accelerate the inferential capabilities of hybrid Transformer-CNN networks. The Convolution and Transformer computations can be mapped to a systolic array by unifying these operations for matrix multiplication. Softmax and LayerNorm which are frequently used in hybrid Transformer-CNN networks were also implemented on FPGA boards. The accelerator achieved high performance with a peak throughput of 722 GOP/s at an average energy efficiency of 53 GOPS/W. Its respective computation latencies were 51.3 ms, 18.1 ms, and 6.8 ms for ViT-Base, ViT-Small, and ViT-Tiny. The accelerator provided a 12× improvement in energy efficiency compared to the CPU, a 2.3× improvement compared to the GPU, and a 1.5× to 2× improvement compared to existing accelerators regarding speed and energy efficiency.

1. Introduction

In recent years, deep learning (LeCun et al., 2015; Menghani, 2023) has made significant strides in various areas including audio signal processing, computer vision, and natural language processing (NLP). Convolutional Neural Networks (CNNs), a type of deep learning architecture, can effectively extract detailed features from images, making them widely applicable in computer vision tasks. Conversely, the transformer's deep learning architecture is based on a multi-head attention mechanism. In addition, transformers have been widely used in NLP tasks, offering tremendous performance improvements over conventional CNN and Recurrent Neural Network (RNN) models. Compared with CNNs, Transformers offer several advantages in computer vision

tasks, such as modeling global visual information, powerful feature representation, scalability, generalization, and the potential for parallel processing. Vision Transformers (ViT) are transformer-based network models that have achieved improved accuracy and efficiency in the field of computer vision (Dosovitskiy et al., 2021).

Artificial Intelligence (AI) accelerators are specialized hardware devices engineered to enhance the speed and efficiency of artificial intelligence applications. Each type of AI accelerator offers distinct advantages and is primarily designed to optimize neural network operations. Before transformers gained prominence, the majority of AI accelerators were mainly focused on accelerating convolutional neural networks. Convolutional accelerators typically provide limited support

* Corresponding author.

E-mail addresses: qingzengsong@tiangong.edu.cn (Q. Song), 2131091172@tiangong.edu.cn (Y. Dai), 2131091203@tiangong.edu.cn (H. Lu), jinguanghao@tjpu.edu.cn (G. Jin).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

<https://doi.org/10.1016/j.jksuci.2024.102194>

Received 8 June 2024; Received in revised form 19 September 2024; Accepted 20 September 2024

Available online 25 September 2024

1319-1578/© 2024 The Authors. Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

for 1×1 and 3×3 convolutions, restricting their flexibility and scalability in accommodating diverse network structures. By converting convolutions into matrix multiplications using `Img2col` (Tao et al., 2021; Wang and Li, 2021), the flexibility of convolutional computations can be significantly increased. The systolic array (Kung and Leiserson, 1979) is a computational architecture characterized by a grid of Processing Elements (PEs) interconnected in a systematic and structured manner. Because of matrix multiplication's inherent parallelism, utilizing a systolic array as a computational engine for these operations is an excellent choice.

Numerous algorithms have been proposed, which hybridize the transformer and convolution architectures (Yuan et al., 2023; Ye et al., 2023a; Song et al., 2023). Hybrid Transformer-CNN models, which integrate CNNs and vision transformers, can at various scales extract joint spectral and spatial features with both local and global information from training samples. However, owing to the significant differences in their computational structure, most accelerators are designed to accelerate either CNNs or transformers separately. Consequently, research on hardware accelerators specifically designed for hybrid Transformer-CNN networks is relatively limited (Li et al., 2023). To address this issue, our study presents a dedicated hardware accelerator specifically designed to accelerate the hybrid Transformer-CNN networks inferring ability. The contributions of this study are summarized as follows.

1. We proposed an efficient systolic array architecture with two distinct computing modes designed to accelerate hybrid Transformer-CNN networks. Our architecture employs specialized mapping methods for both convolution and transformer computations, unifying these operations into a single matrix multiplication process on the systolic array.
2. In this paper, we present an efficient fixed-point hardware implementation of the Softmax and LayerNorm functions, which are frequently used in hybrid Transformer-CNN networks. This innovative approach enhances flexibility and configurability, allowing the accelerator to exploit fully its parallel computing capabilities.
3. The accelerator achieved high performance with a peak throughput of 722 GOP/s and low computational latency for ViT, a network based on transformers that incorporates a convolutional layer as its first layer to partition the input into sub-patches. The accelerator also provides a $12\times$ improvement in energy efficiency compared to a CPU, a $2.3\times$ improvement compared to a GPU, and offers a $1.5\times$ to $2\times$ improvement compared to existing accelerators regarding speed and energy efficiency.

The remainder of this paper is organized as follows. Section 2 covers the Vision Transformer, hybrid Transformer-CNN networks, quantization techniques, and AI accelerators based on systolic arrays, providing a comprehensive background of the key technologies and methods relevant to the study. Section 3 details the systolic array framework, including its architecture and application to map multi-head attention mechanisms and convolution operations. Section 4 explores the hardware implementation of LayerNorm, including its general principles and the specifics of quantized inference. Section 5 discusses the hardware implementation of the Softmax function. Section 6 presents the evaluation and analysis, including the experimental setup, assessment of Transformer-CNN networks, and a comparison with related works. The main conclusions are presented in Section 7.

2. Background and related works

2.1. Vision transformer

ViT is a visual processing model based on transformer architecture, in which the attention mechanism (Vaswani et al., 2017; Tao et al.,

2018; Xi et al., 2020) is crucial. Inspired by the successful application of transformers in NLP tasks, ViT which divides an input image into equal-sized patches and converts each patch into a vector using linear mapping, has achieved significant advancements in computer vision. These vectors, known as patch embeddings, capture the input image's local information. Positional Encoding is then used to integrate the positional information of the patches, allowing the transformer model to recognize the spatial relationships within the input sequence. The Encoder comprises multiple multi-head self-attention (MSA) layers, multi-layer perceptron (MLP) modules, and Layer Normalization (LN). The multi-head self-attention mechanism captures global dependencies among patches, whereas the MLP modules apply nonlinear transformations to the features of each patch. The specifics of MSA and MLP are detailed below.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V \quad (1)$$

The self-attention function is defined in Eq. (1), where Q represents queries, K denotes keys, V indicates values, and d_k represents the dimension of K . Considering a single self-attention operation as one "head", the multi-head self-attention operation is illustrated in Eq. (2).

$$\text{MSA}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_i) \\ \text{where } \text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (2)$$

$$\text{MLP}(x) = \text{Act}(x \cdot W_h + B_h)W_o + B_o \quad (3)$$

The MLP block, defined as Eq. (3), comprises two linear layers with activation functions. The variables W and B correspond to the weights and biases of the hidden and output layers, respectively. The activation function, denoted by Act , can be either GELU or RELU.

The Encoder is represented as Eq. (4).

$$z_0 = [x_{\text{class}}; x_p^1 E; x_p^2 E \dots; x_p^N E] \\ z'_\ell = \text{MSA}(\text{LN}(z_{\ell-1})) + z_{\ell-1} \\ z_\ell = \text{MLP}(\text{LN}(z'_\ell)) + z'_\ell \\ y = \text{LN}(z_L^0) \quad (4)$$

2.2. Hybrid transformer-CNN networks

CNNs are highly efficient at handling local features and spatial information, making them ideal for image processing tasks. However, these methods have a limited ability to capture global dependencies. In contrast, transformers excel at managing long-range dependencies and the global context, particularly in NLP. However, they have high computational complexity and struggle with high-resolution images. By combining these two approaches, hybrid Transformer-CNN networks achieve efficient local feature extraction and processing while enhancing the capture of global information. This synergy boosts the overall performance and applicability of the model.

Recently, an increasing number of researchers have used hybrid Transformer-CNN networks to leverage their respective advantages in tackling various complex tasks. Xiong et al. (2023) proposed a method that combines ViT and a CNN to improve its performance for fine-grained visual classification (FGVC). Arshad et al. (2024) used a hybrid ViT framework and residual 3D CNN models to address the challenges associated with limited labeled data and imbalanced classes. Li et al. (2022a) proposed a hybrid network architecture for defect classification. The CNN first extracts local features from the images, and the transformer encoder then globally models these features to extract abstract and high-level semantic information. Liu et al. (2023) proposed an efficient and complexity-controllable parallel Transformer-CNN mixture (TCM) block that enhanced the overall architecture of image compression models.

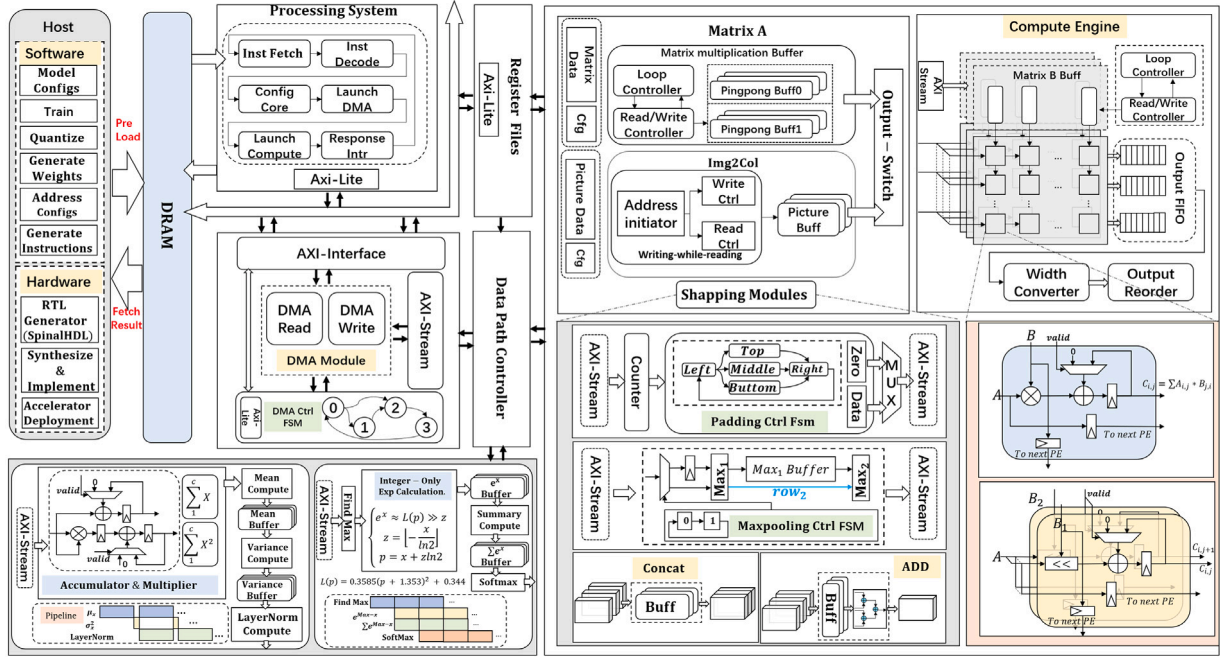


Fig. 1. Overall framework.

2.3. Quantization

Quantization (Stock et al., 2019; Choukroun et al., 2019) involves using fewer bits to represent high-precision floating point data, aiming to quantify the weights of a pre-trained model from floating point values (e.g., float32) to integers with fewer bits (e.g., int8/int4). The model is fine-tuned using a limited amount of calibration data through an inference process. This approach is advantageous for accelerating the inference computation of the model. In deep learning, the two commonly utilized quantization methods are post-training quantization (PTQ) (Liu et al., 2021; Nagel et al., 2020) and quantization-aware training (QAT) (Park et al., 2018; Fan et al., 2020).

Uniform quantization (Widrow et al., 1996) is widely used on most hardware platforms. Eq. (5) describes a quantizer that converts input X into b bits:

$$Q(X|b) = \text{clip} \left(\left\lfloor \frac{X}{S} \right\rfloor + Z_p, 0, 2^b - 1 \right) \quad (5)$$

where S and Z_p are quantization factors determined by the minimum and maximum values of X . S and Z_p can be calculated using the following formula:

$$S = \frac{\max(X) - \min(X)}{2^b - 1} \quad (6)$$

$$Z_p = \text{clip} \left(\left\lfloor \frac{-\min(X)}{\max(X)} \right\rfloor, 0, 2^b - 1 \right) \quad (7)$$

Most published quantization schemes achieve satisfactory results. For example, the transformer training quantization framework LightSeq (Wang et al., 2020), proposed in 2020, supports mixed-precision and quantization-aware training. LightSeq supports various model architectures, including BERT, GPT, Transformer, and ViT, and employs a quantization-aware training strategy. Optimized primarily for GPUs, significant work is still required to adapt its quantization scheme for hardware implementation. In 2021, the post-training quantization strategy FQ-ViT (Lin et al., 2021) introduced per-tensor factor (PTF) quantization for LayerNorm, channel-wise quantization for LayerNorm, and Log-Int-Softmax quantization for Softmax. The primary concept of this quantization strategy involves employing numerical methods to select an appropriate approximate nonlinear function. This approach

ensures that nonlinear computations can be efficiently executed during forward inference while minimizing the loss of precision.

In addition, Q-ViT (Li et al., 2022b) was the first to identify that the bottleneck causing significant performance degradation stems from information distortion in low-quantized self-attention graphs. To address this, an information rectification module (IRM) and a distribution-guided distillation (DGD) scheme for fully quantized vision transformers (Q-ViT) were developed. These innovations effectively eliminate distortion, enabling the successful quantization of ViTs.

2.4. Systolic array-based AI accelerators

A systolic Array (Kung and Leiserson, 1979) is a computational architecture characterized by a grid of PEs interconnected in a systematic and structured manner. The Google Tensor Processing Unit (TPU) (Jouppi et al., 2017) is a specialized Application-Specific Integrated Circuit (ASIC) designed to accelerate deep learning applications. Its primary function involves utilizing a large 256×256 systolic-array-based matrix multiplication engine that supports int8 precision. In 2020, a specialized hardware accelerator (Lu et al., 2020) dedicated to transformers was introduced, featuring a well-designed systolic array to enhance efficient attention-mechanism computation.

Similar to our approach, two nonlinear functions, Softmax and LayerNorm (Xu et al., 2019), were implemented in the hardware. However, the accelerated model BERT was used for natural language processing tasks. In 2023, Ye et al. (2023) utilized the systolic array to accelerate the attention mechanism in transformers and doubled the computational efficiency of the systolic array by constructing a “four processing unit”. Compared with CPUs and GPUs, the computing speed was significantly improved. However, for Softmax, it is also straightforward to convert the exponential base e to exponential base 2. Moreover, a well-designed high-frequency systolic array (Chen et al., 2023) for a FPGA-based transformer accelerator was proposed in 2023. It achieved frequencies of 588 and 474 MHz for different array sizes. Simultaneously, Amirshahi et al. (2023) integrated a tightly coupled systolic array into a computing system to evaluate the benefits of small-scale, tightly coupled systolic arrays for accelerating inference in transformer models. However, the complexity of instructions necessitates additional CPU involvement in the computation.

Algorithm 1 Matrix Multiplication On Systolic Array

Input: Matrix $A_{(H \times W)}$, Matrix $B_{(W' \times W')}$, Height of Systolic Array SA_h , Width of Systolic Array SA_w
Output: Result of Matrix $C_{(H \times W')}$

- 1: Initial size messages of matrices A , B , and C
- 2: Pre-load matrix B into matrix B cache
- 3: **if** initial SA_w columns of matrix B have been loaded **then**
- 4: **for** each i in range $(1, \lceil \frac{H}{SA_h} \rceil)$ **do**
- 5: Load rows $(i-1) \times SA_h + 1$ to $i \times SA_h$ of matrix A
- 6: **for** each j in range $(1, \lceil \frac{W'}{SA_w} \rceil)$ **do**
- 7: **if** columns $(j-1) \times SA_w + 1$ through $j \times SA_w$ of matrix B have already been cached **then**
- 8: Wait $W + SA_w$ cycles for the matrix multiplication of submatrix:
- 9: $Tmp \leftarrow A_{[(i-1) \times SA_h + 1 : i \times SA_h, 1 : \text{end}]} \cdot B_{[1 : \text{end}, (j-1) \times SA_w + 1 : j \times SA_w]}$
- 10: $C_{[(i-1) \times SA_h + 1 : i \times SA_h, (j-1) \times SA_w + 1 : j \times SA_w]} \leftarrow Tmp$
- 11: **end if**
- 12: **end for**
- 13: **end for**
- 14: **end if**
- 15: **Return** C

3. Systolic array**3.1. Overall framework**

In this study, the framework for accelerated hybrid Transformer-CNN networks on FPGA boards as shown in Fig. 1, comprise three main components: the host, processing system, and programmable logic. The host undertakes the training, quantization, weight generation, address configuration, and instruction generation. It ultimately generates RTL source code using SpinalHDL for the hardware accelerator, which can be deployed on the designated hardware. Pre-generated data, including weights, address configurations, and computation instructions, are stored in the DRAM. The processing system (PS) reads instructions from the DRAM and parses them, enabling the configuration of the accelerator core through the AXI-Lite interface. In the DRAM, pre-generated address information contains storage addresses for data, such as images, weights, and computation results. The processing system can also configure the DMA to transfer data from the DRAM to the accelerator core and transfer the processed results back to specific addresses in the DRAM. In summary, the PS primarily handles simple and low-overhead configuration tasks, whereas the programmable logic (PL) side performs intensive computations.

As shown in Fig. 1, the proposed hardware accelerator integrates a configurable three-dimensional systolic array, necessary computation cache units, data-transformation modules, nonlinear operator computation units, and quantization modules. The systolic array serves as the core computational engine of the accelerator and can perform matrix multiplication. The PEs within the systolic array support two computational modes: standard multiplication and shift-based multiplication. Multiple PEs are interconnected in a crisscross fashion, executing parallel multiply-accumulate operations, and providing strong hardware support for high-density computation tasks such as matrix multiplication and convolution. Once the systolic array completes its computation, the results are passed to the quantization module, where they are re-quantized into an 8-bit format, enabling faster low-bitwidth processing in subsequent modules and reducing memory bandwidth usage.

Correspondingly, there are two data cache units for matrix A and B. The Matrix B Cache can also be used as a weight cache unit for convolution calculations. The Matrix A Cache unit receives configuration information from the register table such as matrix dimensions, start signals, and necessary configuration information for convolution calculations, such as convolution kernel size, image size, stride, and input/output channels. To improve efficiency, the matrix multiplication unit also utilizes a ping-pong buffer to cache data, reducing the possibility of computation interruption. The computation process was divided

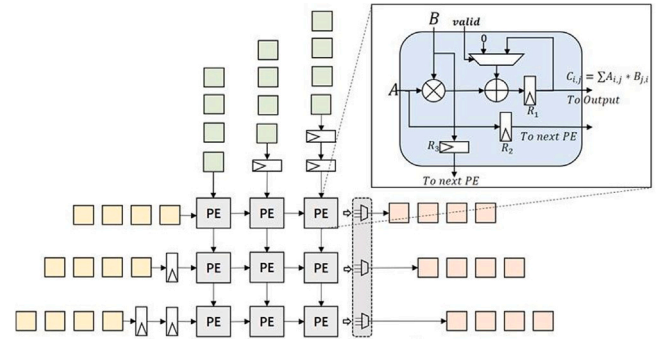


Fig. 2. 2D systolic array.

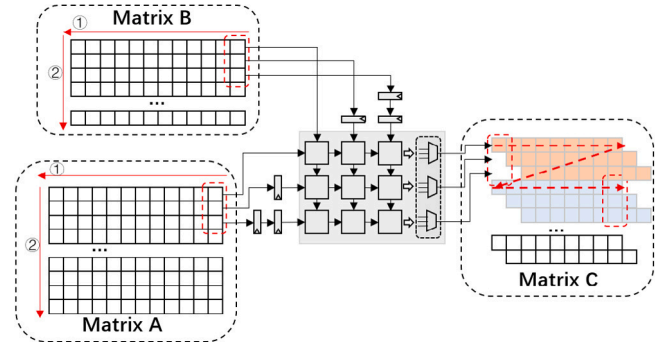


Fig. 3. GEMM on 2D systolic array.

into three stages. In each stage, the engine performs calculations while caching Matrix B's entire data on the chip. The data for Matrix A are partially cached on-chip using ping-pong operations. Matrices A and B are loaded only once from the DRAM. A pipelined approach is used for all computations.

The accelerator employs 8-bit fixed-point computation and supports nonlinear operators, such as LayerNorm and Softmax. In addition, an efficient Img2Col method was designed to map convolutions to matrix multiplications on a systolic array. This method can convert arbitrary three-dimensional convolutions of sizes ranging from 3×3 to 16×16 , with strides ranging from 1 to the kernel size, into matrix multiplication operations on the systolic array. The accelerator incorporated padding and MaxPooling modules commonly used in convolutional neural networks. These modules are specifically designed for CNNs, accelerating

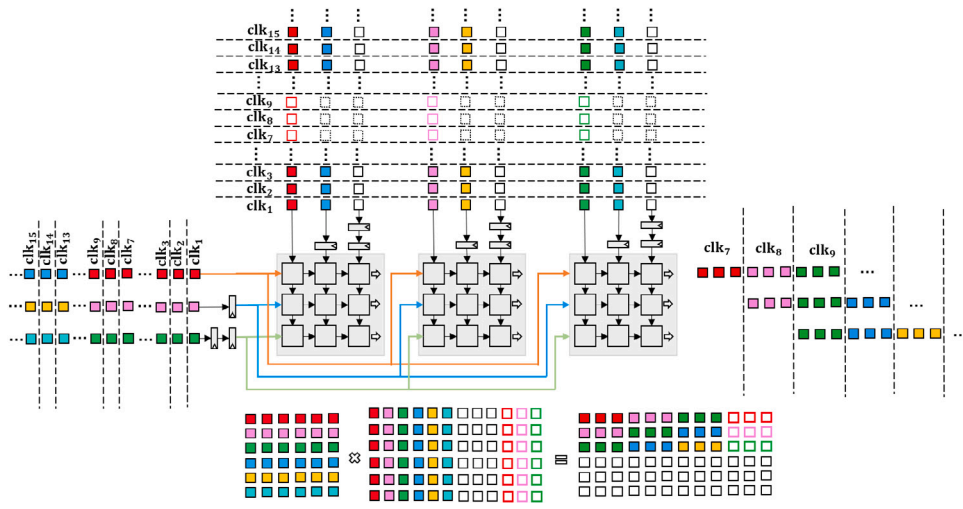


Fig. 4. GEMM on 3D systolic array.

the entire convolution operation at the hardware level, and eliminating the need to rely solely on a CPU or GPU, thereby enhancing the overall processing efficiency.

3.2. Architecture of systolic array

Since the three-dimensional systolic array is an extension of the two-dimensional systolic array, the latter's data architecture must be constructed before implementing the three-dimensional systolic array. The subsequent three-dimensional systolic array can be extended based on the two-dimensional systolic array. Additionally, considering that transformers involve large matrix computations, such as the ultra-large matrix calculations of $[197, 768] \times [768, 3072]$ in ViT-Base, and given that the systolic array can only support limited matrix sizes simultaneously, a reasonable matrix partitioning strategy must be designed.

Fig. 2 shows a systolic array architecture with a fixed output size of 3×3 . The array comprises multiple interconnected PEs, each of which executes a basic multiply-accumulate (MAC) operation. Each PE has an adder-based multiplier augmented with registers R_1 for storing partial and final results, R_2 for matrix B data, and R_3 for matrix A data, as well as a selector controlled by a valid signal to determine whether an accumulation is required. By encapsulating these additional modules, a PE is formed that integrates both storage and computation. To ensure that each element of matrix A's rows and matrix B's columns arrives at the correct PE for the MAC operation at specific cycles, the input data for row i of the systolic array must enter the array one cycle earlier than the data for row $i + 1$. Similarly, the input data for column j of the systolic array must enter the array one cycle earlier than the data for column $j + 1$.

As shown in Fig. 3, matrix A flows from left to right, whereas matrix B flows from top to bottom within the systolic array. Each PE is connected horizontally and vertically to form a stationary output systolic array. In this manner, the data of matrices A and B were input from the left and top of the array, respectively, and each data point in A and B was reused three times. Because the data of matrices A and B flow from one PE to the next adjacent PE in the array rather than being broadcast to all PEs simultaneously, the resulting matrix C is not obtained simultaneously. As shown in Fig. 3, when the PE in the top-left corner of the array completes its calculation, the other PEs have not yet completed their accumulation and computation.

Therefore, the data selector at the output outputs only the results from the top-left PE. In the next clock cycle, the PEs in the first row, second column, second row, and first column complete their calculations simultaneously, and the data selector chooses the results from these PEs for the output. This process continues, and five cycles are

required from the time when the top-left PE completes its calculation to the time when the entire matrix C is fully computed. Considering the data input time, the total computation time for matrix multiplication is the time required for the last data element of matrices A or B to reach the bottom-right PE.

For a two-dimensional systolic array of a size $[SA_t, SA_w]$ the data in matrices A and B are reused SA_w and SA_t times, respectively. The size of the systolic array determines the number of times the data is reused. The larger the systolic array, the larger the matrix size that can be supported for parallel computation. However, simply stacking multiple two-dimensional systolic arrays to construct a three-dimensional systolic array is impractical. This is because deep learning networks comprise multiple layers, and if the output data format of an upper layer does not satisfy the input requirements of the subsequent layer, data rearrangement between each layer is required. This causes an additional resource overhead and significantly increases the computation latency. Therefore, when designing three-dimensional systolic arrays, it is essential to optimize the data flow to ensure the efficient transmission and utilization of data between different layers, thereby reducing the need for data rearrangement. The specific iteration process is described in Algorithm 1.

To ensure consistency in the data input and output formats, the data input and output models must be reconstructed for a three-dimensional systolic array. For illustrative purposes, Fig. 4 visualizes the specific computational process and the data input and output formats of the flattened three-dimensional systolic array. The size of the array is denoted by $[SA_t, SA_h, SA_w]$ (where SA_t represents the number of stacked 2D systolic arrays), and the matrix computations performed on this systolic array are represented by $A \times B = C$. Every SA_h row of matrix A is sequentially computed with every $SA_t \times SA_w$ column of matrix B, producing submatrices with dimensions $[SA_h, SA_t \times SA_w]$.

3.3. Mapping multi-head attention mechanism

The attention mechanism (Vaswani et al., 2017; Tao et al., 2018; Xi et al., 2020) is the most important component of the encoder blocks in ViT. The input to the encoder is a standard two-dimensional matrix. After normalization by LayerNorm, it is used to construct Q , K , and V through the Linear layer. Q , K , and V can be obtained through the linear layer before performing multi-head attention calculations. According to ViT-Tiny, the Linear layer has dimensions of $[197, 192 \times 3]$, which can be considered a matrix formed by concatenating the weights of each head in Q , K , and V :

$$W = [W_1^Q W_2^Q W_3^Q W_1^K W_2^K W_3^K W_1^V W_2^V W_3^V] \quad (8)$$

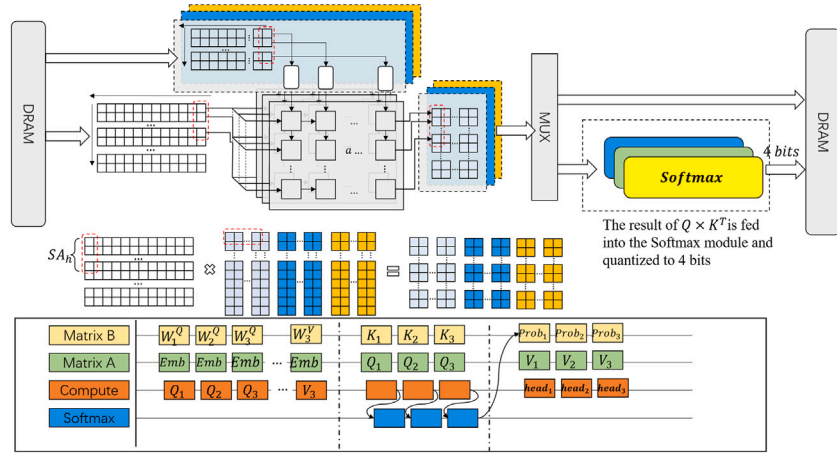


Fig. 5. Pipeline of attention mechanism.

Thus, the computation involves the matrix multiplication of $A \times W$ in the first stage. There are multiple tiles in the systolic array and in Stage 1, each tile performs parallel computations $[Q, K, V] = A \times [W^Q, W^K, W^V]$. The computed results for Q , K , and V must be written back to the DRAM for storage. As shown in Fig. 5, the weight data of the linear layer are used as matrix B, which is the input from the top of the systolic array and Matrix A, which is the result of the embedding layer, is the input from the left of the systolic array. Matrix multiplication is performed between the two matrices. Each data point in matrix A is reused $SA_h \times SA_w$ times, and each data point in matrix B is reused SA_h times. The final calculation result for each head is denoted as $C = [Q_1 Q_2 Q_3 K_1 K_2 K_3 V_1 V_2 V_3]$.

During the final calculation of $\text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \times V$, the PE of the systolic array switches to shift mode. Because Softmax adopts the Log quantization strategy, simple shift operations are sufficient for the remainder of the computation. It is important to note that Attn is ultimately quantized to 4 bits, whereas each PE can receive 8-bit inputs. To optimize bandwidth utilization and minimize wastage, two Shift modules were installed in each PE. Furthermore, it is worth noting that at this stage, the matrix size for the computation was $A_{197 \times 197} \cdot B_{197 \times 64} = C_{197 \times 64}$.

In this particular case, the throughput of the systolic array is limited to only 122.7 GOP/s due to a mismatch of the systolic array, configured as $[Tile, Height, Width] = [4, 8, 64]$, and the column size of matrix B. Consequently, the utilization of PEs becomes significantly lower during the computation. To address this issue, the computation is optimized by swapping the inputs of the systolic array. This allows matrix multiplication $B^T \cdot A^T = C^T$ to be performed on the systolic array, thus achieving an improved throughput of 383.3 GOP/s.

3.4. Mapping convolution

Let the dimension of the input feature be $[B, H, W, C]$ (assuming batch size is 1), shown as Fig. 6, the dimension of the convolution weights be $[C', H, W', C]$, and the dimension of the output feature map be $[B, H', W', C']$, then the input feature map and convolution weights can be expanded into matrix $A_{(W' \times H') \times (K^2 \times C)}$ and $B_{(K^2 \times C) \times C'}$, respectively.

In addition, the final output feature map can also be represented by the matrix $C_{(W' \times H') \times C'}$, which is the result of matrix multiplication $A \times B$. Because the image storage format is $[B, H, W, C]$, the weights of the convolution kernel and each sliding window are expanded along the dimension of the channel, shown as A in Fig. 6.

Each row of matrix A represents the sliding window flattened in the input feature map, each column of matrix B corresponds to the

convolutional weight of each flattened output channel, and each row of matrix C corresponds to all channels of each pixel in the output feature map. For instance, ViT's Embedding layer divides the picture into 14×14 subgraphs through a 16×16 convolution with a step size of 16; each subgraph is input into the encoder as a token after flattening, where each token corresponds to a row in matrix C.

Following Figs. 7 and 8, the convolution is transformed into a matrix multiplication on an $N \times M$ systolic array. Consequently, N convolutional sliding windows are computed simultaneously. Referring to the illustration provided, for ease of explanation the systolic array size is $N \times M$ (where $N = 2$ and $M \leq N$ is assumed). This indicates that the systolic array could process two convolutional sliding windows simultaneously. An in-depth description of how a convolution is expanded into matrix multiplications on a systolic array can be found in Algorithm 2.

4. Hardware implementation of LayerNorm

4.1. LayerNorm

Unlike batch normalization, layer normalization performs the same computation at the training and test times. Eq. (9) is the original formula of LayerNorm (Xu et al., 2019), where the input X is a two-dimensional matrix. The output of LayerNorm is unevenly distributed in each channel, and directly applying quantization methods such as MiniMax similar to convolution to LayerNorm will cause a serious loss of accuracy. Therefore, LayerNorm needs a better quantization method. This study refers to a hardware-friendly quantization method FQ-ViT proposed for 2022. FQ-ViT (Lin et al., 2021) was the first transformer quantization method for the complete quantization of ViT. The accuracy of ViT inference after FQ-ViT quantization was almost unaffected.

$$\text{LayerNorm}(X) = \frac{X - \mu_x}{\sqrt{\sigma_x^2 + \epsilon}} \gamma + \beta, \quad X \in \mathbb{R}^{H \times W} \quad (9)$$

The core idea of PTF is to equip different channels with different factors, rather than different quantization parameters.

$$\begin{cases} S = \frac{\max(X) - \min(X)}{2^b - 1} \\ Z_p = \text{clip}\left(\left\lceil -\frac{\min(X)}{S} \right\rceil, 0, 2^b - 1\right) \\ \alpha_i = \arg \min_{\alpha_i \in \{0, 1, \dots, K\}} \left\| X_i - \left\lceil \frac{X_i}{2^{\alpha_i} S} \right\rceil \times 2^{\alpha_i} S \right\|_2 \end{cases} \quad (10)$$

$$\hat{X}_i = S_{in} \times (Q(X_i | b) - Z_p) \ll \alpha_i = S_{in} \cdot X_Q \quad (11)$$

As shown in the above formula, a global scaling factor S_{in} . Similarly, each channel has its own PTF α_i . After obtaining the quantization factor

Algorithm 2 Img2Col

Input: Image Feature of size $[H, W, C]$, Output image dimensions of $[H', W', C']$, Convolutional kernel of size K , Step of size $S (S \leq K)$, height and width of the systolic array SA_h, SA_w .

Output: A: A 2-D matrix obtained by unfolding the convolution using the img2col algorithm.

```

1: for  $Oh = 1$  to  $H'$  do
2:   Initialization before each iteration
3:    $WI = 1$ 
4:    $WII = 1$ 
5:    $LC = W'$ 
6:   for  $OW = 1$  to  $\lceil \frac{W'}{SA_h} \rceil$  do
7:     for  $Oc = 1$  to  $\lceil \frac{C'}{SA_w} \rceil$  do
8:       for  $K1 = 1$  to  $K$  do
9:         for  $K2 = 1$  to  $K$  do
10:          for  $Ic = 1$  to  $\frac{C}{SA_h}$  do
11:            for  $Sr = 1$  to  $SA_h$  do
12:              OutPut = Feature $_{(K1+RI, WII:WII+SA_h-1)}$ 
13:               $WII = WII + S \times C$ 
14:              if  $Sr == LC$  then
15:                break
16:              end if
17:            end for
18:             $WII \leftarrow WI + Ic \times SA_H + (K2 - 1) \times C$ 
19:          end for
20:           $WII \leftarrow WI + K2 \times C$ 
21:        end for
22:         $WII \leftarrow WI$ 
23:      end for
24:    end for
25:     $LC \leftarrow LC - SA_h$ 
26:     $WI \leftarrow WI + SA_h \times C \times S$ 
27:     $WII \leftarrow WI$ 
28:  end for
29:   $RI \leftarrow RI + S$ 
30: end for

```

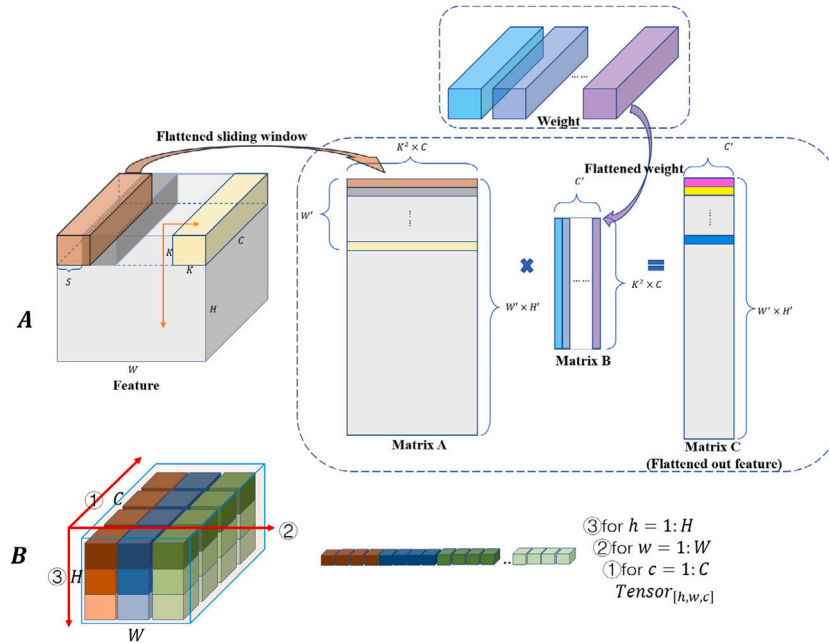


Fig. 6. Visualization of CNN mapped to matrix multiplication and a 3-D tensor is flattened along the channel dimension.

Algorithm 3 LayerNorm PipeLine

Input: Input matrix $A_{h \times w}$, 8-bits Bias data: $Bias_{1 \times w}$, 8-bits Scale data: $Scale_{1 \times w}$, 2-bits Ptf data: $Ptf_{1 \times w}$, and 8-bit zeropoint

Output: Calculation result of LayerNorm: $B = \text{Layernorm}(A)$

```

1: Initial Loop Controller with size of matrix A and B
2: Pre-load  $Scale$  (8 bits  $\times w$ ),  $Bias$  (8 bits  $\times w$ ),  $Zeropoint$  (8 bits  $\times w$ ), and  $Ptf$  (2 bits  $\times w$ )
3: for  $i$  in range 0 to  $h + 1$  do
4:    $M_1^i \leftarrow 0, M_2^i \leftarrow 0$ 
5:   if  $i == 0$  then
6:     for  $j$  in range 1 to  $w$  do
7:        $Tmp_j^i \leftarrow [(A_{1,j} + zeropoint) \ll Ptf_j] \times w$ 
8:        $M_1^i \leftarrow M_1^i + A_{1,j}$ 
9:        $M_2^i \leftarrow M_2^i + (A_{1,j})^2$ 
10:      Cache  $Tmp_j^0$  into storage units of Stage 2 module
11:    end for
12:    Store  $M_1^i, M_2^i$ 
13:  else
14:    for  $j$  in range 1 to  $w$  do
15:
16:       $Tmp_j^i \leftarrow [(A_{1,j} + zeropoint) \ll Ptf_j] \times w$ 
17:       $M_1^i \leftarrow M_1^i + A_{i,j}$ 
18:       $M_2^i \leftarrow (A_{i,j})^2$ 
19:       $RS^{(i-1)} \leftarrow \frac{1}{\sqrt{(w \times M_2^{(i-1)}) \times (M_1^{(i-1)})^2}}$ 
20:
21:       $SA_j^{(i-1)} \leftarrow Scale_j \times Tmp_j^{(i-1)}$ 
22:      Push  $SA$  into FIFO, waiting for the completion of square root calculation in Stage 1
23:       $SAB_j^{(i-1)} = SA_j^{(i-1)} \times RS^{(i-1)} + Bias_j$ 
24:       $B_{i,j} \leftarrow \text{Truncate}(SAB_j^{(i-1)})$ 
25:    end for
26:    Store  $M_1^i, M_2^i$ 
27:  end if
28: end for
return B

```

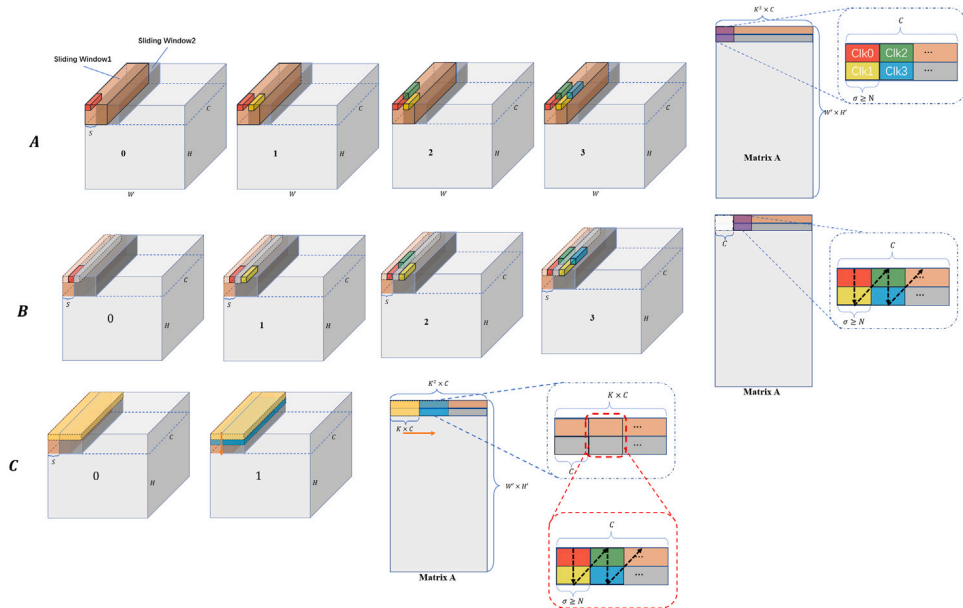


Fig. 7. Visualization of core loops of the Img2Col algorithm: A: The innermost loop, corresponds to step 1 in Fig. 8. B: The second loop, corresponds to step 2. C: The third loop, corresponds to step 3.

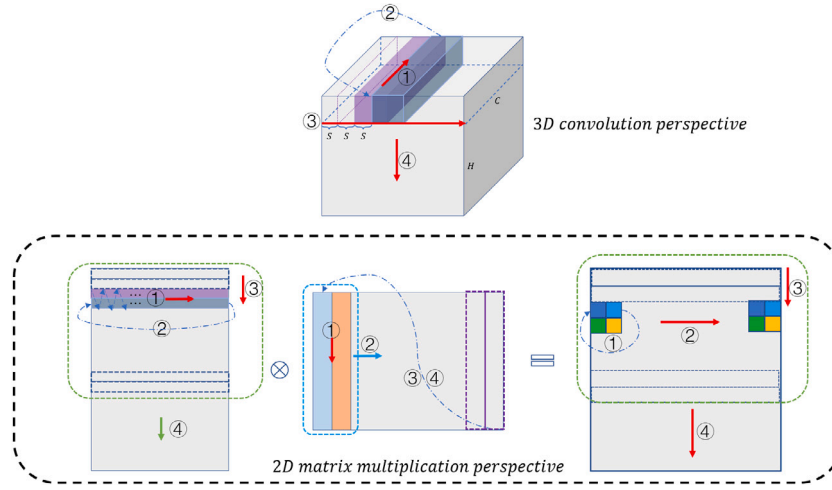


Fig. 8. Visualization of CNN mapping to matrix multiplication.

through PTF, the LayerNorm formula must be further organized to adapt to the hardware calculation. First, the means of X and X^2 are determined as follows:

$$\mu_x \approx \frac{S_{in}}{W} \sum_{i=1}^W X_{Q_i} = \frac{S_{in}}{W} M_1 \quad (12)$$

$$\mu_{x^2} \approx \frac{S_{in}^2}{W} \sum_{i=1}^W X_{Q_i}^2 \rightarrow \frac{S_{in}^2}{W} M_2 \quad (13)$$

Second, we utilize μ_x and μ_{x^2} to calculate σ_x^2 :

$$\sigma_x^2 = \mu_{x^2} - \mu_x^2 \approx \frac{S_{in}^2}{W^2} (CM_2 - M_1^2) \quad (14)$$

Then, Eq. (10) can be written as Eq. (15):

$$\text{LayerNorm}(X) = \frac{S_{in} X_Q - \frac{S_{in}}{W} M_1}{\sqrt{\frac{S_{in}^2}{W^2} (CM_2 - M_1^2)}} \gamma + \beta \quad (15)$$

LayerNorm can be expressed in its simplest form as follows:

$$\text{LayerNorm} = A \times B \times \gamma + \beta \quad (16)$$

To achieve integer-only forward propagation, it must also make a final arrangement of Eq. (16). Let Y_Q be the quantized LayerNorm output, S_{out} and Z_{pout} be the quantization factors of LayerNorm output; then, the statistics of input X based on integer-only calculations can be obtained as follows:

$$Y_Q = \left[A \times B \times \left(\frac{\gamma}{S_{out}} \right) + \frac{\beta}{S_{out}} \right] + Z_{pout} \quad (17)$$

The results for $\frac{\gamma}{S_{out}}$ and $\frac{\beta}{S_{out}} + Z_{pout}$ can be precomputed and quantized as parameters to be passed to the LayerNorm computation module. Therefore, the hardware implementation of LayerNorm must only focus on the fixed-point calculation of $A \times B$ in Eq. (17). Thus, we obtain the formulation for the integer-based forward propagation of LayerNorm, where the calculation of the reciprocal square root $\frac{1}{\sqrt{x}}$ can be achieved using Newton's iterative method. A hardware computational data stream can be designed using a fixed-point forward propagation formula.

4.2. Quantized inference of LayerNorm

The distinction between LayerNorm and BatchNorm lies in how the former's mean and variance are dynamically calculated. This implies that the LayerNorm calculation requires an initial pass through all inputs to determine the mean value of the current input before

proceeding with subsequent computations. Consequently, LayerNorm's calculation flow can be divided into two distinct stages. The initial stage involves preprocessing the input X_q . When loading a row in matrix X , the $W \times X_q$, X_q^2 , M_1 , and M_1^2 values for that row are dynamically and simultaneously computed. In this row, the value of $W \times M_2 - M_1^2$ can be derived, representing the input for a module under a square-root operation.

Furthermore, when calculating $W \times X_q$, the $W \times X_q$ value is stored in the two-stage MEM for subsequent second-stage calculations. Similarly, during the second stage of the computation, the cached value of $W \times X_q$ and the previously computed M_1 must be used to calculate A . Subsequently, A is multiplied by the incoming precomputed parameter scale to yield the result of $Scale \times A$, which then undergoes FIFO caching until the $\frac{1}{\sqrt{x}}$ operation is complete. Simultaneously, Matrix X 's next row continues to load, thereby initiating the first-stage computation for the next row. This also allows the reuse of the same memory space to store the values of $W \times X_q$ for Matrix X 's following rows.

Ultimately, a two-stage pipeline can be designed as depicted in the diagram. The required computation time is determined by $(H + 1) \times (W + \epsilon)$, where ϵ represents the maximum computation delay among all the internal computing units. In the current design, the square-root operation accounts for the maximum delay, which amounts to 33 clock cycles. The detailed hardware architecture and computational flow are respectively illustrated in Fig. 9 and Algorithm 3.

5. Hardware implementation of Softmax

Before computing Softmax (Stevens et al., 2021), all inputs must be iterated through to find the maximum value. Each input is subtracted from this maximum value before contributing to the Softmax calculation. This approach effectively prevents the issue of excessive numerical precision owing to exceedingly large input values. Therefore, the Softmax function can be expressed as follows:

$$\text{Softmax}(x)_i = \frac{e^{(x_i - x_{\max})}}{\sum_{j=1}^k e^{(x_j - x_{\max})}} \quad (18)$$

Eq. (18) describes the modified Softmax formula that adjusts the inputs by subtracting the maximum value, thus ensuring numerical stability during the computation. The i th element of the Softmax output vector is represented by $\text{Softmax}(x)_i$. x_i denotes the i th element of the input vector x . $x_{\max}$ represents the maximum value among all elements of x . The denominator $\sum_{j=1}^k e^{(x_j - x_{\max})}$ represents the sum of the exponentiated differences between each element of x and $x_{\max}$, where k represents the total number of elements in x .

Algorithm 4 SoftMax Pipeline

Input: QK^T calculation result: 2D matrix $A \in \mathbb{R}^{h \times (n-h)}$, quantization factor S_{in} , and the zeropoint. For example, the square matrix $A_{(:,i-h+1:(i+1)-h)}$ represents the computation result of the i -th head of QK^T , where $0 \leq i \leq n-1$, h represents the embedding dimension and n denotes the number of heads.

Output: 4-bit quantized result of Softmax

```

1: for  $i = 1$  to  $n$  do
2:   for  $j = 1$  to  $h+2$  do
3:      $MAX_i^j \leftarrow 0$ 
4:     if  $j = 1$  then
5:       for  $k = 1$  to  $h$  do
6:         if  $MAX_i^1 < A_{1,i-h+k}$  then
7:            $MAX_i^1 \leftarrow A_{1,i-h+k}$ 
8:         end if
9:         Cache  $A_{j,i-h+k}$  in Cache1
10:      end for
11:    else
12:       $Sum_i^j \leftarrow 0$ 
13:      for  $k = 1$  to  $h$  do
14:        if  $1 < j \leq h$  then
15:          if  $MAX_i^j < A_{j,i-h+k}$  then
16:             $MAX_i^j \leftarrow A_{j,i-h+k}$ 
17:          end if
18:           $Temp_{j-1}^h \leftarrow I - \text{Exp}(A_{j-1,i-h+k} - \text{zeropoint} - MAX_i^{j-1}, S_{in})$ 
19:           $Sum_i^{j-1} \leftarrow Sum_i^{j-1} + Temp_j^h$ 
20:           $LGE_j^h \leftarrow \log_2(Temp_j^h)$ 
21:        end if
22:        if  $j > 2$  then
23:           $Shift \leftarrow \log_2(Sum_i^{j-2}) - LGE_{j-2}^h$ 
24:        end if
25:      end for
26:    end if
27:  end for
28: end for

```

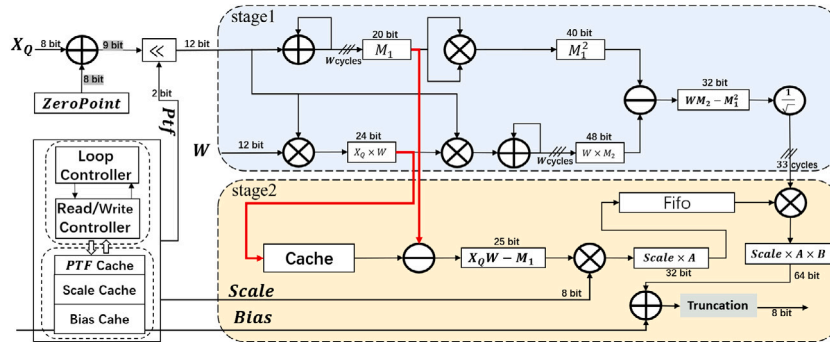


Fig. 9. Hardware implementation of layer normalization.

In a hardware implementation, to design a Softmax computation pipeline, multiple iterations of computing e^x can be performed. However, such calculations are expected to introduce significant delays with unpredictable timing. To address this problem, approximation methods such as i-exp (Kim et al., 2021) can be applied to compute e^x . Because the input x is now nonpositive, any negative number can be decomposed into the form $(-\ln 2) \cdot Z + p$, where Z is a nonnegative number and $p \in (-\ln 2, 0]$. Let the input be $X = -\ln 2 \cdot z + p$ then, we have

$$e^X = e^{-\ln 2 \cdot z + p} = e^p \gg Z \quad (19)$$

Furthermore, we can approximate e^p using the following second-order polynomial:

$$e^p \approx L(p) = 0.3585(p + 1.353)^2 + 0.344 \quad (20)$$

Therefore, $e^X \approx L(p) \gg z$, where $z = \lfloor \frac{X}{-\ln 2} \rfloor$ and $p = X + z \cdot \ln 2$. Next, we simplify the equation to derive the final quantized forward-propagation formula and, based thereon, design a quantized computation pipeline. Because the matrix calculations are performed using uniform quantization, we obtain

$$\hat{X}_i = S_{in} \times (Q(X \mid b) - Zp) = S_{in} \times X_Q \quad (21)$$

as a result,

$$e^X \approx e^{\hat{X}} = e^{S_{in} \cdot X_Q} \approx e^{-\ln 2 \cdot z + p} = L(p) \gg z. \quad (22)$$

$$z = \max \left(\left\lfloor \frac{X_Q S_{in}}{-\ln 2} \right\rfloor, 30 \right) \quad (23)$$

$$p = S_{in} (X_Q + z \cdot \frac{\ln 2}{S_{in}}) = S_{in} \cdot P \quad (24)$$

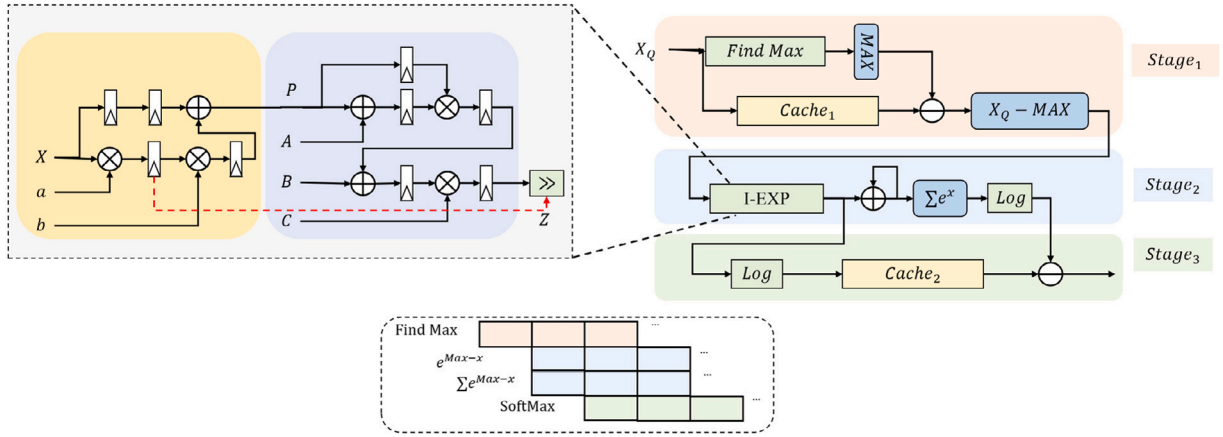


Fig. 10. Softmax hardware implementation pipeline.

$$L(S_{in} \cdot P) = a \cdot S_{in}^2 \left[P \left(P + \frac{b}{a \cdot S_{in}} \right) + \frac{c}{a \cdot S_{in}^2} \right] \quad (25)$$

Because parameters such as S_{in} , $\ln 2$, a , b , and c in Eq. (25) are known, they can be fixed-point quantized and treated as constant values in a hardware implementation. This provides a fully quantized fixed-point calculation for e^x . The I-EXP hardware computation dataflow was designed precisely for this purpose, as illustrated by the I-EXP module presented in Fig. 10.

After obtaining the approximate value of e^x using I-Exp, the cumulative sum of e^x , denoted as $\sum e^x$, must be calculated. Finally, the Softmax result $Attn$ must be quantized using Log-2 quantization with a 4-bit representation. The calculation of Log-2 can be referenced in the log module depicted in Fig. 10. In the final computation of $\text{Softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V = Attn \cdot V$, only shift operations must be executed, as shown in Eq. (26).

$$Attn \cdot V = \frac{S_V}{2^N} \cdot (V_Q - zp_V) \ll (N - Attn) \quad (26)$$

Because of the Softmax calculation process, all inputs must be iterated through to find the maximum value. After obtaining the maximum current value, an exponential calculation of e and the cumulative sum of the exponentials must be performed. Thus, two passes of traversing the input data are required to complete one round of Softmax calculation. To reduce the computation time and storage overhead, we designed a 3-stage data flow similar to the Layernorm pipeline. In Stage 1, the input data are iterated to determine the maximum dynamic value. Stage 2 calculates e^x and the cumulative sum of the exponentials ($\sum e^x$). Simultaneously, the logarithm base 2 of e^x is cached in the stage 3 calculations and together with the precomputed V_Q , the final computation of $\frac{S_V}{2^N} (V_Q - zp_V) \ll (N - Attn)$ is performed. The detailed hardware architecture and computational flow are respectively illustrated in Fig. 10 and Algorithm 4.

6. Evaluation and analysis

We conducted a comprehensive analysis of the accelerator from multiple perspectives, including computational capability, power consumption, and resource utilization.

6.1. Experimental setup

The accelerator was deployed on the ZCU102, which has 274 K logic units (LUTs), 548 K flip-flops (FFs), 2520 DSP slices, and 912 on-chip BRAMs. The systolic array is configured at the maximum size supported by the FPGA design: [Tile, Height, Width] = [4, 8, 64]. The

accelerator operated at a frequency of 200 MHz and was synthesized and implemented using Vivado 2018.3.

It integrates an instruction register table that can be configured through the Axi-lite interface, Img2Col units for convolution computation, GEMM units for matrix computation, nonlinear function computation units, and the necessary cache units. By co-designing the software and hardware, the programmable logic side of the accelerator interacts with the processing system through the AXI protocol.

6.2. Performance evaluation of accelerator

To verify the support for convolutions, multiple tests (ranging from 33 to 16×16) were conducted to examine the convolutions' performance with different strides. The input image size was set to 480 with eight input channels and 256 output channels. By appropriately configuring the register file, the accelerator efficiently completed computation tasks, as shown in Fig. 11.

In this study, ViT-Base, ViT-Small, and ViT-Tiny were used as the target acceleration algorithms. Model weights have two sources: pretrained weights and weights obtained after training on a custom dataset. The weights were pretrained using the ImageNet2012 dataset. Similarly, to further verify the accuracy of the quantization algorithm, three classes were randomly selected locally from the ImageNet2012 dataset as training data to retrain the ViT networks of three different sizes. The quantization strategy employed in this study was the hardware-friendly FQ-ViT (Lin et al., 2021).

As depicted in Fig. 12, the systolic arrays of different sizes were configured for the ViT computation. Notably, the larger systolic array sizes caused the resource overhead to increase while significantly reducing the computation latency. The size of the systolic array is directly correlated with the level of parallelism. Consequently, a higher overall parallelism leads to improved throughput. The power consumption rises from 2.5 W for the (1, 8, 8) configuration to 12 W for the (4, 8, 64) configuration. However, the throughput increases from 25.1 GOP/s to 638 GOP/s, resulting in a 25× improvement. In addition, the energy efficiency rises from 10.9 GOPs/W to 53 GOPs/W.

To evaluate the performance of the proposed accelerator comprehensively, its performance in computing each network layer is presented in Tables 1 and 2. The table evidences that the proposed accelerator exhibits a remarkable performance in supporting ViT networks that incorporate convolutional and attentional mechanisms. The proposed accelerator performed excellently in supporting ViT networks that included convolution and attention mechanisms. For each layer of the network, the accelerator efficiently executes computations and exhibits significant advantages regarding computation time and volume. The accelerator can strongly support large matrix computations. For example, its peak throughput for convolution calculations is 665.6 GOP/s. The

Table 1
ViT-Base performance on ZCU102.

Layer	Operation	Input size	Weight	Output size	MACs (GOP)	Delay (ms)	Throughput (GOP/s)
Embedding	Convolution	[8,224,224]	[768,8,16,16]	[196,768]	0.617	0.901	684.422
	Concat	–	–	[197,768]	–	0	–
	LayerNorm	[197,768]	–	[197,768]	–	0.102	–
Encoder*12	MSA	Linear	[197,768]	[3,768,768]	0.697	0.976	714.45
		$Q \times K^T$	[12,197,64]	[12,197,64]	0.06	0.191	312.92
		Softmax	[12,197,197]	[12,197,197]	–	0.027	–
		$V^T \times \text{attn}^T$	[12,64,197]	[197,768]	0.06	0.156	383.3
		Add	[197,768]	[197,768]	–	0.096	–
	MLP	LayerNorm	[197,768]	[197,768]	–	0.103	–
		Linear+Relu	[197,768]	[197,3072]	0.93	1.29	722.01
		Linear+Add	[197,3072]	[197,768]	0.93	1.339	693.98
	Cls	LayerNorm	[197,768]	[197,768]	–	0.103	–
		Full connection	[1,768]	[1,1000]	0.002	0.058	26.68
Total	–	–	–	–	32.724	51.28	638.20

Table 2
ViT-Small performance on ZCU102.

Layer	Operation	Input size	Weight	Output size	MACs (GOP)	Delay (ms)	Throughput (GOP/s)
Embedding	Convolution	[8,224,224]	[384,8,16,16]	[196,384]	0.308	0.637	483.98
	Concat	–	–	[197,384]	–	0	–
	LayerNorm	[197,384]	–	[197,384]	–	0.052	–
Encoder*12	MSA	Linear	[197,384]	[3,384,384]	0.174	0.3	581.26
		$Q \times K^T$	[6,197,64]	[6,197,64]	0.3	0.095	312.917
		Softmax	[6,197,197]	[6,197,197]	–	0.030	–
		$V^T \times \text{attn}^T$	[6,197,197]	[197,384]	0.03	0.078	383.3
		Add	[197,384]	[197,384]	–	0.049	–
	MLP	LayerNorm	[197,384]	[197,384]	–	0.053	–
		Linear+Relu	[197,384]	[197,1536]	0.232	0.356	653.28
		Linear+Add	[197,1536]	[197,384]	0.232	0.482	482.47
	Cls	LayerNorm	[197,384]	[197,384]	–	0.052	–
		Full connection	[1,384]	[1,1000]	0.001	0.029	26.11
Total	–	–	–	–	8.69	18.09	480.7

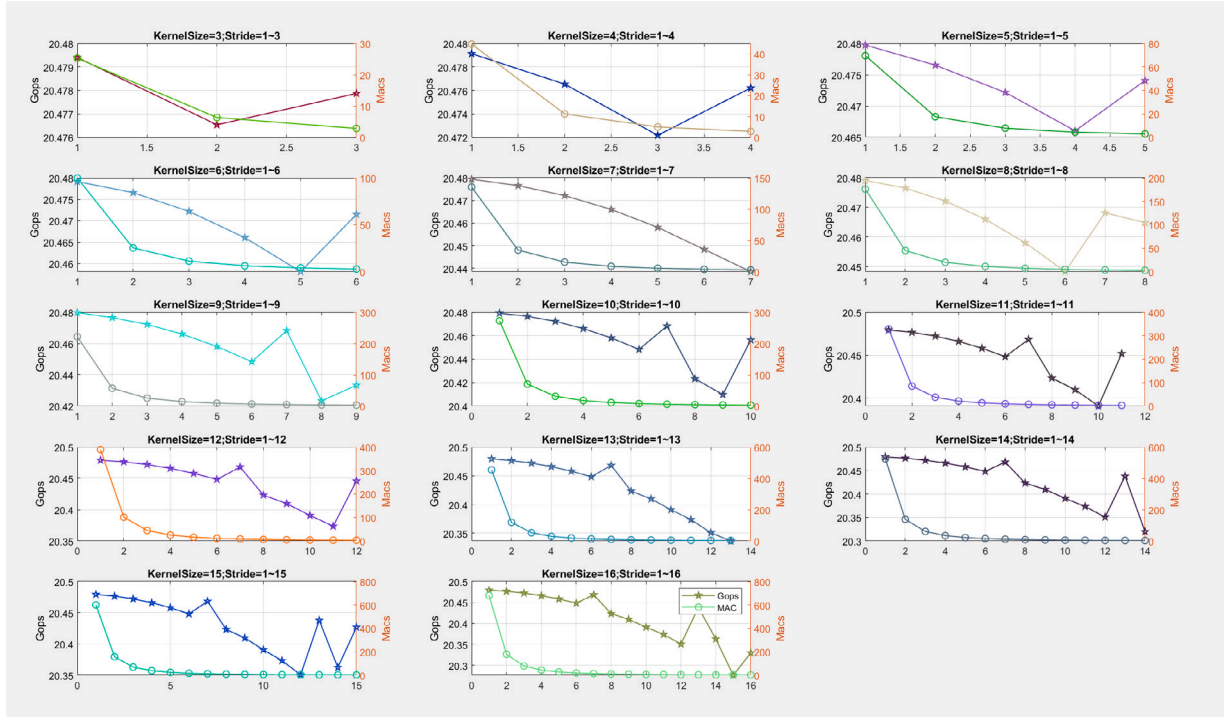


Fig. 11. Support for convolution with different sizes and strides.

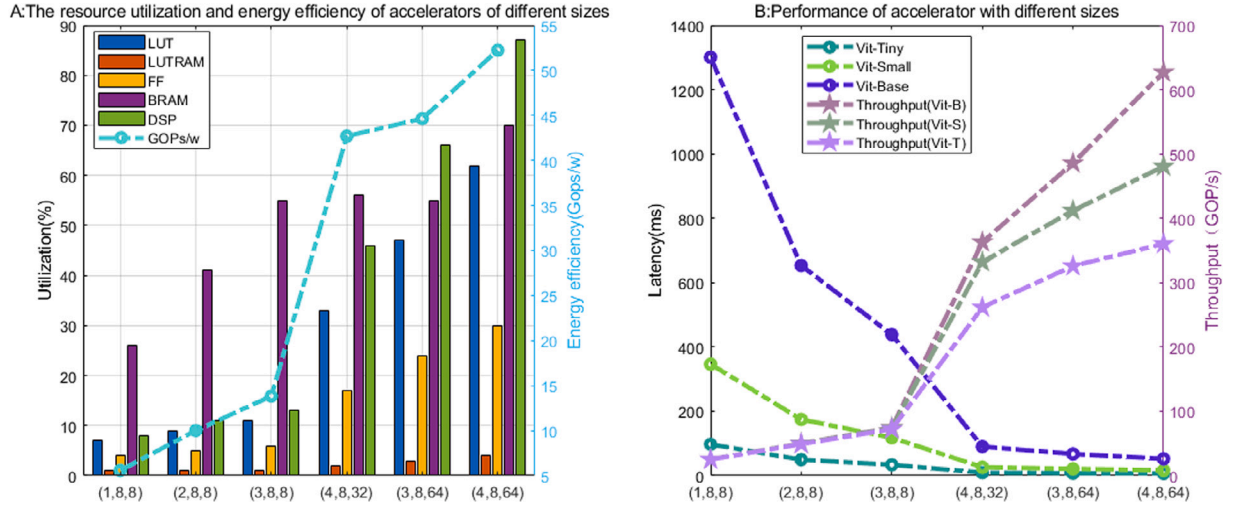


Fig. 12. Comparison of characteristics and resource utilization among accelerators with different sizes.

Table 3

Performance comparison with previous FPGA implementations.

	FPGA platform	Model	Data type	DSP	BRAM	LUT	FF	Power (W)	Throughput (GOP/s)	Latency (ms)	Energy efficiency (GOPS/W)
Yang and Su (2022)	ZCU102 (150M)	Transformer	W8 A8	1024	539	65k	31K	5.48 W	279.8	1.47	51.06
Wang et al. (2022)	Alveo-U50 (300M)	Swin-T	FP16	2420	1002	258k	257	39 W	309.6	–	7.93
Huang et al. (2023)	ZCU102 (300M)	ViT-small	W8 A8	1268	–	144k	168k	29.6 W	762.7	11.15	25.77
		ViT-tiny		1268	–	144k	168k	29.6 W	616.14	4.077	20.81
Zhao et al. (2022)	VCU709 (200M)	TNT-S	–	2048	1024	156K	77K	12.49 W	728.3	14.7	58.31
Li et al. (2023)	XCVU37p (200M)	Transformer-Base	W8A8	1260	448 BRAM	140k	260k	16.9 W	576.52	0.1908	34.11
		ResNet-50			+			17.6 W	590	13.12	33.52
		BotNet-50			64 URAM			17.2 W	583.38	39.58	33.91
Ours	ZCU102 (200M)	ViT-Base	W8 A8	2185	637	169k	166k	12 W	638.2	51.3	53.14
		ViT-small						11.7 W	480.7	18.1	41.1
		ViT-tiny						11.4 W	359.3	6.8	31.5

accelerator achieved high performance with a peak throughput of 722 GOP/s. However, its performance is slightly less impressive for multi-head attention, because it involves a large number of small-matrix computations. Even so, the average throughputs of the accelerator can reach 638.20.4 GOP/s (ViT-Base) and 480.7 GOP/s (ViT-Small).

For each separate network layer, efficient computations were executed by the accelerator, and notable advantages regarding computation time and computational workload were recorded. This further validates the effectiveness and feasibility of the proposed accelerator for computations involving the convolution and attention mechanisms.

6.3. Comparison with related works

To further demonstrate the superiority of the proposed computational architecture, it was comprehensively compared to state-of-the-art transformer accelerators, as shown in Table 3. This exercise found that by optimizing the computation and data flows, the proposed architecture effectively utilizes the processor, memory, and storage resources while it achieves higher computational efficiency and completes equally scaled tasks more efficiently.

We comprehensively compared using the CPU and the GPU as shown in Fig. 13. These accelerators represent the most advanced technologies in the field of Transformer-CNN acceleration and have achieved significant results. The accelerator achieves a similar FPS to the CPU, while the GPU delivers nine times the FPS of the accelerator. However, the accelerator consumes only one-eighth the power of the

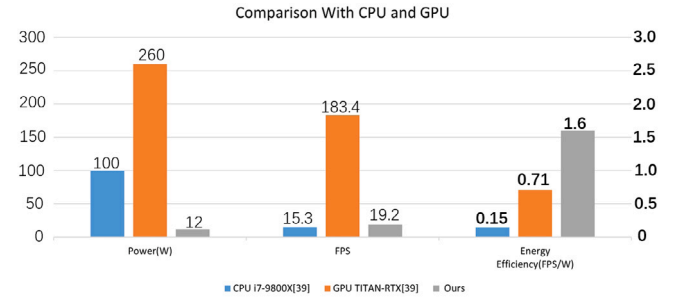


Fig. 13. Comparison with CPU and GPU.

CPU and one-twentieth that of the GPU, demonstrating a 12× improvement in energy efficiency over the CPU and a 2.3× improvement over the GPU.

7. Conclusion

A hardware accelerator was presented in this paper, aimed at enhancing the efficiency and performance of hybrid Transformer-CNN networks on FPGA boards. By leveraging an advanced systolic array architecture and optimizing the calculation flow for attention mechanisms, the overall effectiveness of the accelerator was significantly boosted through this unified solution. Regarding the hardware design

of the accelerator, detailed discussions are provided concerning the hardware implementation of key modules, including the systolic array, Img2Col convolution mapping module, quantization and nonlinear operator module. These design schemes are effective in accelerating the computation process of the Transformer model and enhancing the system's performance.

By mapping convolution operations into general matrix multiplication on systolic arrays, a unified acceleration of both convolution and attention matrix computations can be achieved. This approach, combined with thoughtfully designed computational data flows and the implementation of advanced quantization strategies, ultimately enables the efficient acceleration of the ViT. Overall, the new multi-head attention mechanism calculation strategy for systolic arrays, along with the novel Img2Col algorithm that maps convolution into matrix multiplication on systolic arrays, represents a significant advancement in this field. Additionally, the implementation of Softmax and Layer-Norm on FPGA further enhanced the functionality and versatility of the accelerator. The accelerators can be expanded to include additional application domains.

Certainly, to structure models for more complex tasks, the scalability of hardware accelerators should be further investigated. In the future, this accelerator can be extended to other application levels. Due to its support for both convolution and Transformer operations, it is possible to explore the acceleration of other Hybrid Transformer-CNN Networks, thereby expanding its application from image classification to other scenarios.

CRediT authorship contribution statement

Qingzeng Song: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Yao Dai:** Writing – review & editing, Writing – original draft, Visualization, Validation, Project administration, Methodology, Investigation, Formal analysis, Data curation. **Hao Lu:** Writing – review & editing, Validation, Software, Resources, Investigation. **Guanghao Jin:** Writing – review & editing, Visualization, Validation, Supervision, Resources, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Amirshahi, A., Klein, J.A.H., Ansaloni, G., Atienza, D., 2023. Tic-sat: Tightly-coupled systolic accelerator for transformers. In: *Proceedings of the 28th Asia and South Pacific Design Automation Conference*. pp. 657–663.

Arshad, T., Zhang, J., Ullah, I., 2024. A hybrid convolution transformer for hyperspectral image classification. *Eur. J. Remote Sens.*

Chen, Y., Li, T., Chen, X., Cai, Z., Su, T., 2023. High-frequency systolic array-based transformer accelerator on field programmable gate arrays. *Electronics* 12 (4), 822.

Choukroun, Y., Kravchik, E., Yang, F., Kisilev, P., 2019. Low-bit quantization of neural networks for efficient inference. In: *2019 IEEE/CVF International Conference on Computer Vision Workshop. ICCVW, IEEE*, pp. 3009–3018.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al., 2021. An image is worth 16x16 words: Transformers for image recognition at scale. In: *ICLR*, 2021.

Fan, A., Stock, P., Graham, B., Grave, E., Gribonval, R., Jegou, H., Joulin, A., 2020. Training with quantization noise for extreme model compression. *arXiv preprint arXiv:2004.07320*.

Huang, M., Luo, J., Ding, C., Wei, Z., Huang, S., Yu, H., 2023. An integer-only and group-vector systolic accelerator for efficiently mapping vision transformer on edge. *IEEE Trans. Circuits Syst. I. Regul. Pap.*

Jouppi, N.P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., et al., 2017. In-datacenter performance analysis of a tensor processing unit. In: *Proceedings of the 44th Annual International Symposium on Computer Architecture*. pp. 1–12.

Kim, S., Gholami, A., Yao, Z., Mahoney, M.W., Keutzer, K., 2021. I-bert: Integer-only bert quantization. In: *International Conference on Machine Learning*. PMLR, pp. 5506–5518.

Kung, H.T., Leiserson, C.E., 1979. Systolic arrays (for vlsi). In: *Sparse Matrix Proceedings 1978. Vol. 1, Society for industrial and applied mathematics*, Philadelphia, PA, USA, pp. 256–282.

LeCun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. *Nature* 521 (7553), 436–444.

Li, S., Wu, C., Xiong, N.N., 2022a. Hybrid architecture based on CNN and transformer for strip steel surface defect classification. *Electronics*.

Li, Z., Yang, T., Wang, P., Cheng, J., 2022b. Q-ViT: Fully differentiable quantization for vision transformer. *arXiv preprint arXiv:2201.07703*.

Li, T., Zhang, F., Fan, X., Shen, J., Guo, W., Cao, W., 2023. Unified accelerator for attention and convolution in inference based on fpga. In: *2023 IEEE International Symposium on Circuits and Systems. ISCAS, IEEE*, pp. 1–5.

Lin, Y., Zhang, T., Sun, P., Li, Z., Zhou, S., 2021. Fq-ViT: Post-training quantization for fully quantized vision transformer. *arXiv preprint arXiv:2111.13824*.

Liu, J., Sun, H., Katto, J., 2023. Learned image compression with mixed transformer-CNN architectures. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. CVPR 2023*, pp. 14388–14397.

Liu, Z., Wang, Y., Han, K., Zhang, W., Ma, S., Gao, W., 2021. Post-training quantization for vision transformer. *Adv. Neural Inf. Process. Syst.* 34, 28092–28103.

Lu, S., Wang, M., Liang, S., Lin, J., Wang, Z., 2020. Hardware accelerator for multi-head attention and position-wise feed-forward in the transformer. In: *2020 IEEE 33rd International System-on-Chip Conference. SOCC, IEEE*, pp. 84–89.

Menghani, G., 2023. Efficient deep learning: A survey on making deep learning models smaller, faster, and better. *ACM Comput. Surv.* 55 (12), 1–37.

Nagel, M., Amjad, R.A., Van Baalen, M., Louizos, C., Blankevoort, T., 2020. Up or down? adaptive rounding for post-training quantization. In: *International Conference on Machine Learning*. PMLR, pp. 7197–7206.

Park, E., Yoo, S., Vajda, P., 2018. Value-aware quantization for training and inference of neural networks. In: *Proceedings of the European Conference on Computer Vision. ECCV*, pp. 580–595.

Song, P., Yu, Y., Zhang, Y., 2023. Tth-net: Two-stage transformer-cnn hybrid network for leaf vein segmentation. *Appl. Sci.* 13 (19), 11019.

Stevens, J.R., Venkatesan, R., Dai, S., Khailany, B., Raghunathan, A., 2021. Softmax: Hardware software co-design of an efficient softmax for transformers. In: *2021 58th ACM/IEEE Design Automation Conference. DAC, IEEE*, pp. 469–474.

Stock, P., Joulin, A., Gribonval, R., Graham, B., Jegou, H., 2019. And the bit goes down: Revisiting the quantization of neural networks. *arXiv preprint arXiv:1907.05686*.

Tao, C., Gao, S., Shang, M., Wu, W., Zhao, D., Yan, R., 2018. Get the point of my utterance! learning towards effective responses with multi-head attention mechanism. In: *IJCAI*. pp. 4418–4424.

Tao, W.Z., Wang, Y., Zhang, H., 2021. Overview of tensor layout in modern neural network accelerator. In: *2021 18th International Computer Conference on Wavelet Active Media Technology and Information Processing. ICCWAMTIP, IEEE*, pp. 368–371.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need. *Adv. Neural Inf. Process. Syst.* 30.

Wang, T., Gong, L., Wang, C., Yang, Y., Gao, Y., Zhou, X., Chen, H., 2022. Via: A novel vision-transformer accelerator based on fpga. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 41 (11), 4088–4099.

Wang, B., Li, M., 2021. A structure to effectively prepare the data for sliding window in deep learning. In: *2021 IEEE 6th International Conference on Signal and Image Processing. ICSIP, IEEE*, pp. 1025–1018.

Wang, X., Xiong, Y., Wei, Y., Wang, M., Li, L., 2020. Lightseq: A high performance inference library for transformers. *arXiv preprint arXiv:2010.13887*.

Widrow, B., Kollar, I., Liu, M.-C., 1996. Statistical theory of quantization. *IEEE Trans. Instrum. Meas.* 45 (2), 353–361.

Xi, C., Lu, G., Yan, J., 2020. Multimodal sentiment analysis based on multi-head attention mechanism. In: *Proceedings of the 4th International Conference on Machine Learning and Soft Computing*. pp. 34–39.

Xiong, S., Tzafas, G., Kasaei, H., 2023. Enhancing fine-grained 3D object recognition using hybrid multi-modal vision transformer-CNN models. In: *IEEE International Conference on Intelligent Robots and Systems. IROS 2023*, pp. 5751–5757.

Xu, J., Sun, X., Zhang, Z., Zhao, G., Lin, J., 2019. Understanding and improving layer normalization. *Adv. Neural Inf. Process. Syst.* 32.

Yang, X., Su, T., 2022. Efa-trans: An efficient and flexible acceleration architecture for transformers. *Electronics* 11 (21), 3550.

Ye, D., Ni, Z., Wang, H., Zhang, J., Wang, S., Kwong, S., 2023a. Csformer: Bridging convolution and transformer for compressive sensing. *IEEE Trans. Image Process.*

Ye, W., Zhou, X., Zhou, J., Chen, C., Li, K., 2023. Accelerating attention mechanism on fpgas based on efficient reconfigurable systolic array. *ACM Trans. Embedded Comput. Syst.* 22 (6), 1–22.

Yuan, J., Zhou, F., Guo, Z., Li, X., Yu, H., 2023. Hcformer: hybrid cnn-transformer for l1c1 image denoising. *J. Digit. Imaging* 36 (5), 2290–2305.

Zhao, Z., Cao, R., Un, K.-F., Yu, W.-H., Mak, P.-I., Martins, R.P., 2022. An fpga-based transformer accelerator using output block stationary dataflow for object recognition applications. *IEEE Trans. Circuits Syst. II* 70 (1), 281–285.